LaTeX math aliases

$\mathbf{A}, \mathbf{B}, \ldots, \mathcal{D}, \mathbb{R}, \boldsymbol{\sigma}$

OPEN IN COLAB

Midterm 2 Review (ECE 490/590, Fall 2026)

Review questions with answers, converted from:

1. Probabilistic perspective (Part A)
2. AutogradNumpy (Part B)

Part A: Probabilistic perspective

The steps of the *optimization perspective* of machine learning.

Answer

1. Collect a training dataset $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{y}_1), \ldots, (\mathbf{x}_n, \mathbf{y}_n)\}$.
2. Select a model $\hat{\mathbf{y}} = f(\mathbf{x}; \mathbf{w})$ with weights (parameters) $\mathbf{w}$.
3. Select a loss function $l(\mathbf{y}_i, \hat{\mathbf{y}}_i)$.
4. Train: minimize the average loss to find the optimal weights,

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} l(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{w}))}_{L(\mathcal{D}; \mathbf{w})}.$$

5. For a new input $\mathbf{x}^* \notin \mathcal{D}$, predict $\hat{\mathbf{y}}^* = f(\mathbf{x}^*; \mathbf{w}^*)$.

For linear and other "simple" models this works fine. In general it is not enough: it says nothing about how the training data relates to the data the model will see after deployment.

Q3. When is a random variable Z_1 independent of a random variable Z_2 ($Z_1 \perp Z_2$)? Give three equivalent conditions. What is the i.i.d. assumption?

Answer

$Z_1 \perp Z_2$ if any one of these equivalent conditions holds:

1. $\mathbb{P}(Z_1, Z_2) = \mathbb{P}(Z_1)\mathbb{P}(Z_2)$
2. $\mathbb{P}(Z_1|Z_2) = \frac{\mathbb{P}(Z_1, Z_2)}{\mathbb{P}(Z_2)} = \mathbb{P}(Z_1)$
3. $\mathbb{P}(Z_2|Z_1) = \frac{\mathbb{P}(Z_1, Z_2)}{\mathbb{P}(Z_1)} = \mathbb{P}(Z_2)$

The i.i.d. (independent and identically distributed) assumption says the data samples are mutually independent and are all drawn from the same distribution $\mathbb{P}(X, Y)$.

Q4. Given the dataset

$\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$, a model $\hat{y}_i = f(\mathbf{x}_i; \theta)$, and a loss function $l(y_i, \hat{y}_i)$, show that the following optimization problem can be interpreted as **maximum likelihood estimation**. In the process, show where the i.i.d. assumption is needed. List any other assumptions the interpretation needs.

$$\theta^* = \arg \min_\theta \sum_{i=1}^{n} l(y_i, f(\mathbf{x}_i; \theta))$$

Answer

Treat $\mathbf{x}_i$ and y_i as random vectors. **Assume** the probability of a sample is the exponential of the negative loss:

$$P((\mathbf{x}_i, y_i)|\theta) = \frac{1}{Z} \exp\left(-l(y_i, f(\mathbf{x}_i; \theta))\right),$$

where the normalizer Z does not depend on θ (also an assumption).

If the samples are i.i.d., the probability of the whole dataset is the product of the sample probabilities:

$$P(\mathcal{D}|\theta) = \prod_{i=1}^{n} P((\mathbf{x}_i, y_i)|\theta) = \prod_{i=1}^{n} \frac{1}{Z} \exp\left(-l(y_i, f(\mathbf{x}_i; \theta))\right).$$

A product of exponentials is the exponential of the sum of their powers:

$$P(\mathcal{D}|\theta) = \frac{1}{Z^n} \exp\left(-\sum_{i=1}^{n} l(y_i, f(\mathbf{x}_i; \theta))\right) = \frac{1}{Z^n} \exp\left(-L(\mathcal{D}; \theta)\right),$$

where $L(\mathcal{D}; \theta) = \sum_{i=1}^{n} l(y_i, f(\mathbf{x}_i; \theta))$.

The original problem is $\theta^* = \arg \min_\theta L(\mathcal{D}; \theta)$. Because $\exp(-y)$ is monotonically decreasing, this equals

$$\theta^* = \arg \max_\theta \exp(-L(\mathcal{D}; \theta)) = \arg \max_\theta P(\mathcal{D}|\theta),$$

which is the maximum likelihood estimate.

Q5. Given the dataset

$\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$, a model $\hat{y}_i = f(\mathbf{x}_i; \theta)$, a regularizer $R(\theta)$ and a loss function $l(y_i, \hat{y}_i)$, show that the following optimization problem can be interpreted as **maximum-a-posteriori (MAP) estimation**. Show where the i.i.d. assumption is needed and list any other assumptions.

$$\theta^* = \arg \min_\theta \sum_{i=1}^{n} l(y_i, f(\mathbf{x}_i; \theta)) + \lambda R(\theta),$$

where $\lambda > 0$ balances the loss and the regularizer.

How can regularization be interpreted as an application of Bayes' theorem?

Answer

As in the previous question, assume $P((\mathbf{x}_i, y_i)|\theta) = \frac{1}{Z}\exp(-l(y_i, f(\mathbf{x}_i; \theta)))$ and that the samples are **i.i.d.**, so that

$$P(\mathcal{D}|\theta) = \frac{1}{Z^n}\exp\left(-L(\mathcal{D}; \theta)\right), \qquad L(\mathcal{D}; \theta) = \sum_{i=1}^{n} l(y_i, f(\mathbf{x}_i; \theta)).$$

The problem is $\theta^* = \arg \min_\theta L(\mathcal{D}; \theta) + \lambda R(\theta)$. Taking $\exp(-\cdot)$, which is monotonically decreasing, turns it into a maximization:

$$\theta^* = \arg \max_\theta \exp(-L(\mathcal{D}; \theta)) \exp(-\lambda R(\theta)).$$

The first factor is proportional to the likelihood $P(\mathcal{D}|\theta)$. **Assume** the second factor is the prior:

$$P(\theta) = \frac{1}{Z'}\exp(-\lambda R(\theta)).$$

Then

$$\theta^* = \arg \max_\theta P(\mathcal{D}|\theta)P(\theta).$$

By Bayes' theorem, $P(\mathcal{D}|\theta)P(\theta) = P(\theta|\mathcal{D})P(\mathcal{D})$, so

$$\theta^* = \arg \max_\theta P(\theta|\mathcal{D})P(\mathcal{D}).$$

The evidence $P(\mathcal{D})$ does not depend on θ, so it can be dropped:

$$\theta^* = \arg \max_\theta P(\theta|\mathcal{D}),$$

which is maximum a posteriori estimation. The regularizer is the negative log of the prior on the weights. For example, $R(\theta) = ||\theta||^2$ corresponds to a Gaussian prior.

Q6.

Given two random variables X and Y, specify the relationship between $P(X|Y)$ and $P(Y|X)$ using Bayes' rule.

Answer

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

Q7.

You want to find the weights $\mathbf{W}$ of a neural network trained on the CIFAR-10 dataset $\mathcal{D}$. Define the prior, the likelihood and the posterior in terms of $\mathbf{W}$ and $\mathcal{D}$. Write Bayes' rule to get the posterior from the likelihood and the prior.

Answer

- **Prior** $P(\mathbf{W})$: the probability of the weights $\mathbf{W}$ before seeing the dataset.
- **Likelihood** $P(\mathcal{D}|\mathbf{W})$: the probability of observing the dataset for a particular choice of weights $\mathbf{W}$.
- **Posterior** $P(\mathbf{W}|\mathcal{D})$: the probability of the weights given the dataset $\mathcal{D}$.

By Bayes' rule:

$$P(\mathbf{W}|\mathcal{D}) = \frac{P(\mathcal{D}|\mathbf{W})P(\mathbf{W})}{P(\mathcal{D})}$$

Q8.

What does the *No Free Lunch theorem* (Wolpert, 1996) say? What is the problem of *under-determination*?

Answer

Under-determination: a finite sample of data can be described by quite different models. For example, a straight line and an oscillating curve can both pass through the same points.

No Free Lunch: the data collected so far says nothing about the values of the function in regions you have not observed. Averaged over all possible target functions, every learning algorithm has the same expected performance off the training set. An algorithm can only beat others on a restricted subclass of functions (for example, convex functions) whose structure it exploits. In other words, every learner's success depends on its assumptions (inductive bias) matching the problem.

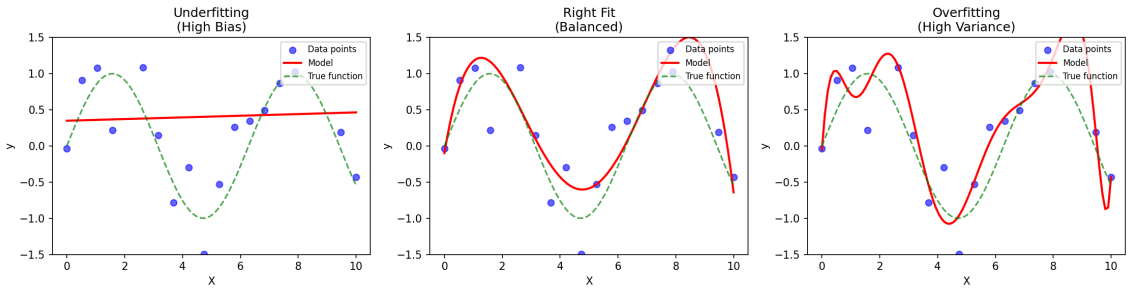
Q9.

What are *underfitting* and *overfitting*? How do you tell them apart using training error and evaluation error?

Answer

- **Underfitting** (high bias): the model is too simple to capture the pattern in the data. Both training error and evaluation error are high.
- **Overfitting** (high variance): the model fits even the noise in the training data instead of ignoring it. Training error is very low but evaluation error is high.
- **A good fit** ignores the noise but captures most of the desired pattern. Evaluation error is low.

In the polynomial-regression example from class (15 noisy points from $3 + x + 2\sin(9x)$), degree 4 underfits, degree 12 fits well, and degree 30 overfits.



Ways to avoid overfitting.

Answer

1. Add a **regularizer** to the loss to prefer "simpler" solutions, e.g. for least squares

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2.$$

2. Use a **validation set** to choose hyper-parameters (e.g. model size, λ) and to decide **when to stop training** (early stopping, when validation loss stops decreasing).
3. Train with **(batch) SGD**: the random choice of small batches keeps the optimizer from fitting the training set exactly.
4. Collect more (representative) training data, or use a smaller model.

Q11. Find the minimum point of the regularized least-squares (ridge regression) problem

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2,$$

where $\mathbf{w} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$, $\mathbf{X} \in \mathbb{R}^{m \times n}$ and $\lambda \in \mathbb{R}^+$.

Answer

Let $f(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|^2$. Write it with inner products:

$$f(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda\mathbf{w}^\top\mathbf{w}.$$

Expand and collect terms:

$$f(\mathbf{w}) = \mathbf{w}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_n)\mathbf{w} - 2\mathbf{y}^\top\mathbf{X}\mathbf{w} + \mathbf{y}^\top\mathbf{y}.$$

Take the derivative (row-vector convention):

$$\frac{\partial}{\partial\mathbf{w}} f(\mathbf{w}) = 2\mathbf{w}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_n) - 2\mathbf{y}^\top\mathbf{X}.$$

At the minimum the derivative is zero:

$$2\mathbf{w}^{*\top}(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_n) - 2\mathbf{y}^\top\mathbf{X} = \mathbf{0}_n^\top.$$

Transpose and solve:

$$\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_n)^{-1}\mathbf{X}^\top\mathbf{y}.$$

$\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_n$ is always invertible for $\lambda > 0$.

Q12.

Find the minimum-norm solution of an under-determined linear system ($\mathbf{X} \in \mathbb{R}^{m\times n}$, $m < n$, full row rank):

$$\mathbf{w}^* = \arg\ \min_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 \text{ s.t. } \mathbf{y} = \mathbf{X}\mathbf{w}.$$

Use the Lagrangian $\frac{1}{2}\|\mathbf{w}\|^2 + \Lambda^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$.

Answer

Set the derivatives of the Lagrangian with respect to $\mathbf{w}$ and Λ to zero:

$$\mathbf{w} - \mathbf{X}^\top\Lambda = 0, \qquad \mathbf{y} - \mathbf{X}\mathbf{w} = 0.$$

Substitute the first into the second:

$$\mathbf{y} - \mathbf{X}\mathbf{X}^\top\Lambda = 0 \implies \Lambda = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{y}.$$

Therefore

$$\mathbf{w}^* = \mathbf{X}^\top(\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{y}.$$

Q13.

Define the L-p norm of $\mathbf{x} \in \mathbb{R}^n$ for $p \in \{1, 2, \ldots\}$.

Answer

$$\|\mathbf{x}\|_p = (|x_1|^p + |x_2|^p + \cdots + |x_n|^p)^{\frac{1}{p}}$$

Q14.

Given i.i.d. random variables $Z_1, Z_2, \ldots, Z_n$ with distribution $\mathbb{P}(\cdot)$, write the mathematical definition of the mean and of the expectation. What is the difference?

Answer

The (sample) **mean** is an average over the observed samples:

$$\bar{Z} = \frac{1}{n}\sum_{i=1}^{n} Z_i.$$

The **expectation** is a probability-weighted average over all possible values of the random variable:

$$\mathbb{E}[Z] = \sum_{z\in\Omega} z\,\mathbb{P}(Z=z) \quad (\text{discrete}), \qquad \mathbb{E}[Z] = \int_{\Omega} z\, f_Z(z)\, dz \quad (\text{continuous}).$$

The mean is computed from data and is itself a random variable. The expectation is a fixed property of the distribution, which is usually unknown. Under the i.i.d. assumption, the mean converges to the expectation as $n \to \infty$ (law of large numbers).

Q15.

What is the difference between *empirical risk* and *expected risk*?

Answer

Empirical risk is the average loss over the dataset. For a dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ and predictions $\hat{y}_i = f(\mathbf{x}_i; \mathbf{W})$:

$$R_{\mathcal{D}}(\mathbf{W}) = \frac{1}{n}\sum_{i=1}^{n} loss(\hat{y}_i, y_i).$$

Expected risk is the expectation of the loss when inputs and outputs are random variables. Assume $\mathcal{D}$ was sampled i.i.d. from $\mathbb{P}_{X,Y}$, where $\mathbf{x}_i$ is an instance of X and y_i an instance of Y:

$$R(\mathbb{P}_{X,Y}, \mathbf{W}) = \mathbb{E}_{X,Y}[loss(\hat{Y}, Y)], \qquad \hat{Y} = f(X; \mathbf{W}).$$

For continuous X, Y this is an integral weighted by the joint PDF:

$$R(\mathbb{P}_{X,Y}, \mathbf{W}) = \int_{x\in\Omega_X}\int_{y\in\Omega_Y} loss(\hat{y}, y)\, f_{X,Y}(x,y)\, dx\, dy, \qquad \hat{y} = f(x; \mathbf{W}).$$

We can compute the empirical risk, but the expected risk is what we actually want to minimize.

Q18.

Why do we split a dataset into training, validation and test data? What is each part used for? What is *data contamination*?

Answer

We can never compute the expected risk $R(\mathbb{P}_{X,Y}, \mathbf{W}) = \mathbb{E}_{X,Y}[loss(\hat{Y}, Y)]$, so we settle for the next best thing: held-out data.

1. **Training data** $\mathcal{D}_t$: to train the model by gradient descent or other algorithms.
2. **Validation data** $\mathcal{D}_v$: to choose hyper-parameters, detect **overfitting**, and decide when to stop training.
3. **Test/evaluation data** $\mathcal{D}_e$: only for evaluating and comparing algorithms. The test data is sacred.

If training/validation data contains any information from the test data, the training data is **contaminated**. This is a big problem for Large Language Models: companies crawl the internet for training data and often end up including the benchmark answers.

If the three splits are sampled uniformly from the collected dataset $\mathcal{D}$, they are likely to be identically distributed. Then the test performance is a good estimate of real-world performance.

Q19.

Write the gradient descent algorithm for minimizing the average training loss $\mathcal{L}(\mathcal{D}_t; \mathbf{W})$.

Answer

given a starting point $\mathbf{W} \in \mathrm{dom} f$.

repeat

1. Choose step size α_t.
2. Update:

$$\mathbf{W} \leftarrow \mathbf{W} - \alpha_t \nabla_{\mathbf{W}} \underbrace{\left(\frac{1}{|\mathcal{D}_t|} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_t} l(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{W})) \right)}_{\mathcal{L}(\mathcal{D}_t; \mathbf{W})} = \mathbf{W} - \frac{\alpha_t}{|\mathcal{D}_t|} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_t} \nabla_{\mathbf{W}} l(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{W}$$

until the stopping criterion is satisfied.

Q20.

Write the Batch Stochastic Gradient Descent (Batch-SGD) algorithm. What is the algorithm called when the batch size $|\mathcal{B}_j| = 1$?

Answer

given a starting point $\mathbf{W} \in \mathrm{dom} f$.

for e **in** range(n_epochs):

1. Choose step size α_t.
2. Shuffle the training dataset $\mathcal{D}_t$.
3. For each small batch $\mathcal{B}_j \subset \mathcal{D}_t$:

$$\mathbf{W} \leftarrow \mathbf{W} - \frac{\alpha_t}{|\mathcal{B}_j|} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{B}_j} \nabla_{\mathbf{W}} l(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{W})).$$

4. Evaluate the validation loss $\frac{1}{|\mathcal{D}_v|} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_v} l(\mathbf{y}_i, f(\mathbf{x}_i; \mathbf{W}))$. Stop when it stops decreasing.

With batch size $|\mathcal{B}_j| = 1$, the algorithm is called Stochastic Gradient Descent (SGD).

Q21.

What are the advantages of Batch-SGD over gradient descent (GD)? When should you use plain GD?

Answer

Advantages of Batch-SGD:

1. **Faster on big datasets.** GD can make one update only after looping over the entire dataset. A uniformly sampled small batch gives roughly the same information as the full dataset, so SGD makes progress much sooner.
2. **Better aligned with the ML objective.** In ML the true objective is the *expected* loss, not the *empirical* training loss. The random choice of small batches keeps Batch-SGD from overfitting to the training set.

Use GD:

1. For optimization problems that are not machine learning.
2. With small datasets and simple models, when you know the model you want to fit (e.g. polynomial regression).

Q22.

You are doing 0-1 binary classification on MNIST, where detecting the digit 1 is the positive class. Using this example, define False negative, False positive, Accuracy, Precision, Recall and F1-score.

Answer

- **False negative:** the classifier predicts negative (0) but the digit is actually 1.
- **False positive:** the classifier predicts positive (1) but the digit is actually 0.
- **Accuracy** = (number of correct classifications) / (total number of samples).
- **Precision** = true positives / predicted positives = $\frac{TP}{TP+FP}$.
- **Recall** = true positives / actual positives = $\frac{TP}{TP+FN}$.
- **F1-score** = harmonic mean of precision and recall = $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}}$.

Q23.

What is a ROC (Receiver Operating Characteristic) curve for a binary classifier? Argue why it lies above the diagonal for any reasonable classifier. Also argue why it must be concave.

Answer

The ROC curve plots the true positive rate $TPR = P(\hat{Y}(X) = 1|Y = 1)$ against the false positive rate $FPR = P(\hat{Y}(X) = 1|Y = 0)$ as the decision threshold varies.

Above the diagonal. A trivial classifier can predict the positive class with probability $\alpha \in (0, 1)$ without looking at X. It has $FPR = TPR = \alpha$, which traces the diagonal. Any classifier better than this trivial one lies above the diagonal.

Concave. Take two operating points on the ROC curve: classifier $\hat{Y}_1$ with $FPR = f_1$, $TPR = TPR(f_1)$ and classifier $\hat{Y}_2$ with $FPR = f_2$, $TPR = TPR(f_2)$. Build a randomized classifier $\hat{Y}$ that uses $\hat{Y}_1$ with probability α and $\hat{Y}_2$ with probability $1 - \alpha$. Then

$$\begin{aligned} P(\hat{Y}(X) = 1|Y = 1) &= P(\hat{Y}_1(X) = 1|Y = 1)\alpha + P(\hat{Y}_2(X) = 1|Y = 1)(1 - \alpha) \\ &= \alpha\, TPR(f_1) + (1 - \alpha)\, TPR(f_2), \end{aligned}$$

and similarly its $FPR = \alpha f_1 + (1 - \alpha) f_2$. So any point on the chord between two ROC points is achievable. The ROC curve is the best achievable TPR at each FPR, so it lies on or above every chord:

$$TPR(\alpha f_1 + (1 - \alpha) f_2) \geq \alpha\, TPR(f_1) + (1 - \alpha)\, TPR(f_2).$$

That is the definition of concavity: g is concave if $g(\alpha y_1 + (1 - \alpha) y_2) \geq \alpha g(y_1) + (1 - \alpha) g(y_2)$ for all y_1, y_2 and $\alpha \in (0, 1)$.

Q24.

What is the likelihood ratio test for a binary classifier? Is it better or worse than the posterior ratio test?

Answer

The likelihood is the probability of the evidence $\mathbf{x}$ given a class $y \in \{0, 1\}$: $\mathbb{P}(\mathbf{x}|y)$. The likelihood ratio test predicts the positive class when the likelihood ratio exceeds a threshold η:

$$\hat{y}(\mathbf{x}) = \begin{cases} 1 & \text{if } \frac{\mathbb{P}(\mathbf{x}|y=1)}{\mathbb{P}(\mathbf{x}|y=0)} \geq \eta \\ 0 & \text{otherwise} \end{cases}$$

The posterior is $\mathbb{P}(y|\mathbf{x}) = \frac{\mathbb{P}(\mathbf{x}|y)\mathbb{P}(y)}{\mathbb{P}(\mathbf{x})}$, and the posterior ratio test is

$$\hat{y}(\mathbf{x}) = \begin{cases} 1 & \text{if } \frac{\mathbb{P}(y=1|\mathbf{x})}{\mathbb{P}(y=0|\mathbf{x})} \geq \eta_p \\ 0 & \text{otherwise} \end{cases}$$

Neither is better: they are equivalent. The posterior ratio condition can be rewritten as a likelihood ratio condition:

$$\frac{\mathbb{P}(y=1|\mathbf{x})}{\mathbb{P}(y=0|\mathbf{x})} \geq \eta_p \iff \frac{\mathbb{P}(\mathbf{x}|y=1)\mathbb{P}(y=1)}{\mathbb{P}(\mathbf{x}|y=0)\mathbb{P}(y=0)} \geq \eta_p \iff \frac{\mathbb{P}(\mathbf{x}|y=1)}{\mathbb{P}(\mathbf{x}|y=0)} \geq \frac{\mathbb{P}(y=0)}{\mathbb{P}(y=1)}\eta_p.$$

This is the likelihood ratio test with $\eta = \frac{\mathbb{P}(y=0)}{\mathbb{P}(y=1)}\eta_p$.

Part B: Automatic differentiation with scalars

Three ways to compute derivatives on a computer. Briefly compare them.

Answer

1. **Numerical differentiation:** finite differences, e.g. $f'(x) \approx \frac{f(x+h)-f(x)}{h}$. Easy, but approximate (truncation and round-off error) and needs one extra function evaluation per input dimension.
2. **Symbolic differentiation:** manipulate the expression to get an exact formula for the derivative (as in Mathematica/SymPy). Exact, but the expressions can blow up in size and it does not work directly on arbitrary programs (loops, branches).
3. **Automatic differentiation:** apply the chain rule to the elementary operations that the program actually executes. Exact up to floating point and efficient. It has two modes:
   A. Forward mode: propagates derivatives from the input side to the output side, alongside the values.
   B. Reverse mode (backpropagation): first computes the values, then propagates derivatives from the output side back to the inputs.

Part C: Chain rule, VJPs and automatic differentiation in NumPy

Q32. (Will not be asked in exam)

Derive the scalar single-variable chain rule $\frac{\partial f(g(x))}{\partial x} = f'(g(x))g'(x)$ from the limit definition of the derivative.

Answer

From $g'(x) = \lim_{h\to 0} \frac{g(x+h)-g(x)}{h}$ we get $\lim_{h\to 0} g(x+h) = \lim_{h\to 0} g(x) + g'(x)h$. Then

$$\begin{aligned}
\frac{\partial f(g(x))}{\partial x} &= \lim_{h\to 0} \frac{f(g(x+h)) - f(g(x))}{h} \\
&= \lim_{h\to 0} \frac{f(g(x) + g'(x)h) - f(g(x))}{h} \\
&= \lim_{h\to 0} \frac{f(g(x)) + f'(g(x))g'(x)h - f(g(x))}{h} \\
&= f'(g(x))g'(x).
\end{aligned}$$

Q33. (Will not be asked in exam)

Derive the chain rule for a function of two variables,

$\frac{\partial}{\partial x} f(u(x), v(x)) = ?$.

Answer

$$\frac{\partial f(u(x), v(x))}{\partial x} = \lim_{h\to 0} \frac{f(u(x+h), v(x+h)) - f(u(x), v(x))}{h} = \lim_{h\to 0} \frac{f(u + u'(x)h, v + v'(x)h) -}{h}$$

Expand $f(u + \delta u, v + \delta v)$ in two steps, first in u and then in v:

$$f(u + \delta u, v + \delta v) \approx f(u, v + \delta v) + f'_u(u, v + \delta v)\delta u \approx f(u, v) + f'_v(u, v)\delta v + f'_u(u, v + \delta v)\delta u.$$

Since $f'_u(u, v + \delta v)\delta u = f'_u(u, v)\delta u + f''_{uv}(u, v)\,\delta v\,\delta u$ and the second-order term vanishes in the limit,

$$f(u + \delta u, v + \delta v) \approx f(u, v) + f'_v(u, v)\delta v + f'_u(u, v)\delta u.$$

With $\delta u = u'(x)h$ and $\delta v = v'(x)h$:

$$\frac{\partial f(u(x), v(x))}{\partial x} = f'_u(u(x), v(x))\,u'(x) + f'_v(u(x), v(x))\,v'(x).$$

Q34. (Will not be asked in exam)

Let $f(\mathbf{g}) : \mathbb{R}^m \to \mathbb{R}$ and $\mathbf{g}(x) : \mathbb{R} \to \mathbb{R}^m$. Using the row-vector (numerator-layout) convention, what are the shapes of $\frac{\partial f}{\partial \mathbf{g}}$ and $\frac{\partial \mathbf{g}}{\partial x}$? Write the chain rule for $\frac{\partial}{\partial x} f(\mathbf{g}(x))$. Does the order of multiplication matter?

Answer

$\frac{\partial f}{\partial \mathbf{g}} = \begin{bmatrix} \frac{\partial f}{\partial g_1} & \dots & \frac{\partial f}{\partial g_m} \end{bmatrix}$ is a $1 \times m$ row vector. $\frac{\partial \mathbf{g}}{\partial x} = \begin{bmatrix} \frac{\partial g_1}{\partial x} & \dots & \frac{\partial g_m}{\partial x} \end{bmatrix}^\top$ is an $m \times 1$ column vector.

$f(\mathbf{g}(x)) = f(g_1(x), \dots, g_m(x))$ is a multivariate scalar function, so by the multivariable chain rule

$$\frac{\partial}{\partial x} f(\mathbf{g}(x)) = f'_{g_1}(\mathbf{g}(x)) g'_1(x) + \cdots + f'_{g_m}(\mathbf{g}(x)) g'_m(x) = \begin{bmatrix} f'_{g_1} & \dots & f'_{g_m} \end{bmatrix} \begin{bmatrix} g'_1(x) \\ \vdots \\ g'_m(x) \end{bmatrix} = \frac{\partial f}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial x}.$$

The order matters: $\frac{\partial \mathbf{g}}{\partial x} \frac{\partial f}{\partial \mathbf{g}}$ would be an $m \times m$ outer product, not the derivative. With a column-vector convention the order would be reversed.

Q35.

State the general chain rule for $\mathbf{f}(\mathbf{g}(\mathbf{x}))$ where $\mathbf{f} : \mathbb{R}^m \to \mathbb{R}^n$ and $\mathbf{g} : \mathbb{R}^p \to \mathbb{R}^m$. What are the Jacobian shapes?

Answer

$$\underbrace{\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{g}(\mathbf{x}))}_{n \times p} = \underbrace{\frac{\partial \mathbf{f}}{\partial \mathbf{g}}}_{n \times m} \underbrace{\frac{\partial \mathbf{g}}{\partial \mathbf{x}}}_{m \times p}.$$

Q36.

How many floating point operations does it take to multiply $A \in \mathbb{R}^{m \times n}$ by $B \in \mathbb{R}^{n \times p}$?

Answer

$C = AB$ has mp entries. Each entry is a dot product of a row of A and a column of B, $\mathbf{a}_i^\top \mathbf{b}_j = a_{i1} b_{1j} + \cdots + a_{in} b_{nj}$, which takes n multiplications and $n - 1$ additions.

Total: $mp(2n - 1)$ operations, i.e. $O(mnp)$.

Q37.

Consider $\mathbf{h}(\mathbf{x}) : \mathbb{R}^m \to \mathbb{R}^n$, $\mathbf{g}(\mathbf{h}) : \mathbb{R}^n \to \mathbb{R}^p$ and $\mathbf{f}(\mathbf{g}) : \mathbb{R}^p \to \mathbb{R}^q$, so that

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{g}(\mathbf{h}(\mathbf{x}))) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{x}}.$$

(a) What is the computational complexity of forward-mode and of reverse-mode differentiation? (b) Which is cheaper for $m = 1000$, $n = 100$, $p = 10$, $q = 1$? (c) What happens for a long chain of k functions?

Answer

The Jacobians have shapes $\frac{\partial \mathbf{f}}{\partial \mathbf{g}} : q \times p$, $\frac{\partial \mathbf{g}}{\partial \mathbf{h}} : p \times n$ and $\frac{\partial \mathbf{h}}{\partial \mathbf{x}} : n \times m$.

(a) **Forward mode** multiplies from the input side (right to left):

$$\frac{\partial \mathbf{f}}{\partial \mathbf{g}} \left(\frac{\partial \mathbf{g}}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right) : \quad O(pnm) + O(qpm) = O\big((pn + qp)\, m\big).$$

Reverse mode multiplies from the output side (left to right):

$$\left(\frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{h}} \right) \frac{\partial \mathbf{h}}{\partial \mathbf{x}} : \quad O(qpn) + O(qnm) = O\big(q\,(pn + nm)\big).$$

(b) Forward: $10 \cdot 100 \cdot 1000 + 1 \cdot 10 \cdot 1000 = 1{,}010{,}000$. Reverse: $1 \cdot 10 \cdot 100 + 1 \cdot 100 \cdot 1000 = 101{,}000$. Reverse mode is about 10 times cheaper.

(c) For a chain of Jacobians $J_k \cdots J_2 J_1$ where J_i has shape $q_i \times p_i$ ($p_i = q_{i-1}$), forward mode costs $O\big((\sum_{i \geq 2} q_i p_i)\, p_1\big)$: the input size p_1 multiplies every term. Reverse mode costs $O\big(q_k \sum_{i<k} q_i p_i\big)$: the output size q_k multiplies every term. So forward mode is cheaper when there are few inputs, and reverse mode is cheaper when there are few outputs.

Q38.

Why is reverse-mode differentiation (backpropagation) the standard method for training neural networks?

Answer

We differentiate the loss function, which is a scalar: its output dimension is $q = 1$. Reverse-mode cost is proportional to the output dimension, and forward-mode cost is proportional to the input dimension (the number of weights, often millions). One reverse pass gives the gradient with respect to all weights, so reverse mode is clearly superior for loss-function gradients. In neural networks, reverse-mode differentiation is called backpropagation.

Q39.

What is a vector-Jacobian product (VJP)? Write the template of a `vjp` function for a function `g(arg1, arg2)` used in reverse mode. Why is it called a VJP, and why don't we always compute the Jacobian explicitly?

Answer

The output of the network is a scalar loss l, so the left-most factor in the reverse-mode chain rule is a row vector:

$$\frac{\partial}{\partial \mathbf{x}} l(\mathbf{f}(\mathbf{g}(\mathbf{x}))) = \frac{\partial l}{\partial \mathbf{f}} \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{x}}.$$

A `vjp` function for $\mathbf{g}$ receives the accumulated row vector $\frac{\partial l}{\partial \mathbf{g}}$ and returns $\frac{\partial l}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \arg}$ for each argument: a row **vector** times the local **Jacobian**.

It helps to imagine a scalar loss $l(\mathbf{g}(\mathbf{x}))$ whose gradient $\frac{\partial l}{\partial \mathbf{g}}$ is given. You don't have to build the Jacobian separately: it is often large and sparse (e.g. diagonal for element-wise operations), and the product can be computed much more cheaply directly.

Q40.

What is a Jacobian-vector product (JVP) used in forward mode? Write the template of a `jvp` function. How do you compute $\frac{\partial \mathbf{f}}{\partial x_2}$ in forward mode, and how many passes does each mode need to compute the full Jacobian of $\mathbf{f} : \mathbb{R}^m \to \mathbb{R}^q$?

Answer

Assume the input is a function of a scalar t, $\mathbf{x}(t)$. Then

$$\frac{\partial}{\partial t} \mathbf{f}(\mathbf{g}(\mathbf{h}(\mathbf{x}(t)))) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial t}.$$

A `jvp` function multiplies the local Jacobian by the incoming column vector of derivatives with respect to t:

To get $\frac{\partial \mathbf{f}}{\partial x_2}$, seed $\frac{\partial \mathbf{x}}{\partial t} = \mathbf{e}_2 = [0, 1, 0, \ldots, 0]^\top$.

Forward mode gives one **column** of the Jacobian per pass, so it needs m passes (one per input). Reverse mode gives one **row** per pass, so it needs q passes (one per output).

Q43.

Find the VJP of element-wise addition $\mathbf{f}(\mathbf{a}, \mathbf{b}) = \mathbf{a} + \mathbf{b}$, where $\mathbf{a}, \mathbf{b}, \mathbf{f} \in \mathbb{R}^n$. That is, given $\frac{\partial l}{\partial \mathbf{f}}$, find $\frac{\partial l}{\partial \mathbf{a}}$ and $\frac{\partial l}{\partial \mathbf{b}}$.

Answer

$$\frac{\partial l}{\partial \mathbf{a}} = \frac{\partial l}{\partial \mathbf{f}} \frac{\partial}{\partial \mathbf{a}} (\mathbf{a} + \mathbf{b}) = \frac{\partial l}{\partial \mathbf{f}} (\mathbf{I}_{n \times n} + 0_{n \times n}) = \frac{\partial l}{\partial \mathbf{f}}, \qquad \frac{\partial l}{\partial \mathbf{b}} = \frac{\partial l}{\partial \mathbf{f}}.$$

```
def add_vjp(dldf, a, b):
    return unbroadcast(a, dldf), unbroadcast(b, dldf)
```

Q44.

Find the VJP of multiplication $f(\alpha, \beta) = \alpha\beta$ (scalars, or element-wise for arrays).

Answer

$$\frac{\partial l}{\partial \alpha} = \frac{\partial l}{\partial f}\beta, \qquad \frac{\partial l}{\partial \beta} = \frac{\partial l}{\partial f}\alpha.$$

```
def mul_vjp(dldf, a, b):
    return unbroadcast(a, dldf * b), unbroadcast(b, dldf * a)
```

Q45.

Find the VJP of the vector dot product $f(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$, where $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ and $f \in \mathbb{R}$.

Answer

$$\frac{\partial l}{\partial \mathbf{a}} = \frac{\partial l}{\partial f}\frac{\partial}{\partial \mathbf{a}}(\mathbf{a}^\top \mathbf{b}) = \frac{\partial l}{\partial f}\mathbf{b}^\top, \qquad \frac{\partial l}{\partial \mathbf{b}} = \frac{\partial l}{\partial f}\mathbf{a}^\top.$$

Q46.

Find the VJP of matrix-vector multiplication $\mathbf{f}(\mathbf{A}, \mathbf{b}) = \mathbf{A}\mathbf{b}$, where $\mathbf{f} \in \mathbb{R}^m$, $\mathbf{b} \in \mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{m \times n}$. Define $\frac{\partial l}{\partial \mathbf{A}}$ to have the same shape as $\mathbf{A}$, with entries $\frac{\partial l}{\partial a_{ij}}$.

Answer

Write $\mathbf{A}$ in terms of its rows $\mathbf{a}_i^\top$, so that $\mathbf{A}\mathbf{b} = \begin{bmatrix}\mathbf{a}_1^\top \mathbf{b} & \dots & \mathbf{a}_m^\top \mathbf{b}\end{bmatrix}^\top$.

Each $f_i = \mathbf{a}_i^\top \mathbf{b}$ is a scalar that depends only on row i of $\mathbf{A}$. So $\frac{\partial f_i}{\partial \mathbf{A}}$ is an $m \times n$ matrix whose i-th row is $\mathbf{b}^\top$ and whose other rows are zero. With $\frac{\partial l}{\partial \mathbf{f}} = \begin{bmatrix}\frac{\partial l}{\partial f_1} & \dots & \frac{\partial l}{\partial f_m}\end{bmatrix}$, row i of $\frac{\partial l}{\partial \mathbf{A}}$ is $\frac{\partial l}{\partial f_i}\mathbf{b}^\top$:

$$\frac{\partial l}{\partial \mathbf{A}} = \begin{bmatrix}\frac{\partial l}{\partial f_1}\mathbf{b}^\top \\ \vdots \\ \frac{\partial l}{\partial f_m}\mathbf{b}^\top\end{bmatrix} = \left(\frac{\partial l}{\partial \mathbf{f}}\right)^\top \mathbf{b}^\top = \left(\mathbf{b}\frac{\partial l}{\partial \mathbf{f}}\right)^\top \in \mathbb{R}^{m \times n}.$$

The derivative with respect to $\mathbf{b}$ is simpler:

$$\frac{\partial l}{\partial \mathbf{b}} = \frac{\partial l}{\partial \mathbf{f}}\frac{\partial}{\partial \mathbf{b}}(\mathbf{A}\mathbf{b}) = \frac{\partial l}{\partial \mathbf{f}}\mathbf{A} \in \mathbb{R}^{1 \times n}.$$

Q47.

Find the VJP of matrix-matrix multiplication $\mathbf{F}(\mathbf{A}, \mathbf{B}) = \mathbf{AB}$, where $\mathbf{F} \in \mathbb{R}^{m \times p}$, $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$. Check your answer by matching shapes.

Answer

Write $\mathbf{B}$ and $\mathbf{F}$ in terms of their columns: $\mathbf{f}_i = \mathbf{A}\mathbf{b}_i$ for $i = 1, \ldots, p$. From the matrix-vector VJP, $\frac{\partial l}{\partial \mathbf{f}_i} \frac{\partial \mathbf{f}_i}{\partial \mathbf{A}} = \left(\mathbf{b}_i \frac{\partial l}{\partial \mathbf{f}_i}\right)^\top$. Since l depends on $\mathbf{A}$ through all the columns $\mathbf{f}_1, \ldots, \mathbf{f}_p$, the multivariable chain rule sums over them:

$$\frac{\partial l}{\partial \mathbf{A}} = \sum_{i=1}^{p} \left(\mathbf{b}_i \frac{\partial l}{\partial \mathbf{f}_i}\right)^\top = \left(\begin{bmatrix} \mathbf{b}_1 & \cdots & \mathbf{b}_p \end{bmatrix} \begin{bmatrix} \frac{\partial l}{\partial \mathbf{f}_1} \\ \vdots \\ \frac{\partial l}{\partial \mathbf{f}_p} \end{bmatrix} \right)^\top = \left(\mathbf{B}\left(\frac{\partial l}{\partial \mathbf{F}}\right)^\top\right)^\top = \frac{\partial l}{\partial \mathbf{F}} \mathbf{B}^\top.$$

For $\mathbf{B}$, apply the same rule to $\mathbf{F}^\top = \mathbf{B}^\top \mathbf{A}^\top$ and transpose:

$$\frac{\partial l}{\partial \mathbf{B}} = \mathbf{A}^\top \frac{\partial l}{\partial \mathbf{F}}.$$

Shape check: $\frac{\partial l}{\partial \mathbf{F}}$ is $m \times p$, so $\frac{\partial l}{\partial \mathbf{F}} \mathbf{B}^\top$ is $(m \times p)(p \times n) = m \times n$, the shape of $\mathbf{A}$. $\mathbf{A}^\top \frac{\partial l}{\partial \mathbf{F}}$ is $(n \times m)(m \times p) = n \times p$, the shape of $\mathbf{B}$. Matching shapes is a quick way to remember these formulas.

Q48.

Write the `vjp` functions for the element-wise operations `exp`, `log`, `maximum(a, b)` (ReLU is `maximum(x, 0)`), and for `sum(x, axis)`. All functions take the incoming gradient `dldf` as the first argument.

Answer

- $\frac{d}{dx} e^x = e^x$, $\frac{d}{dx} \log x = 1/x$.
- `maximum` passes the gradient to whichever argument is larger.
- `sum` copies the incoming gradient to every summed element, so it is broadcast back to the shape of `x`.

In [3]:
```python
# Code not to be written in exam
import numpy as np

def exp_vjp(dldf, x):
    return (unbroadcast(x, dldf * np.exp(x)),)

def log_vjp(dldf, x):
    return (unbroadcast(x, dldf / x),)

def maximum_vjp(dldf, a, b):
    dlda = dldf * np.where(a > b, 1, 0)
    dldb = dldf * np.where(a > b, 0, 1)
    return unbroadcast(a, dlda), unbroadcast(b, dldb)
```

```
def sum_vjp(dldf, x, axis=None, **kwargs):
    if axis is not None:
        dldx = np.expand_dims(dldf, axis=axis) * np.ones_like(x)
    else:
        dldx = dldf * np.ones_like(x)
    return (unbroadcast(x, dldx),)
```

Q49. (Not for exam)

What is the `unbroadcast` function for? Suppose `A.shape = (100, 1)`, `B.shape = (1, 100)` and `F = A + B`. What are the shapes of `dl__dF`, `dl__dA` and `dl__dB`, and how do you compute `dl__dA` and `dl__dB` from `dl__dF`? Also, what will the code below print?

In [4]:
```
import numpy as np

def unbroadcast(target, g, axis=0):
    """Remove broadcasted dimensions by summing along them."""
    while np.ndim(g) > np.ndim(target):
        g = g.sum(axis=axis)
    for axis, size in enumerate(target.shape):
        if size == 1:
            g = g.sum(axis=axis, keepdims=True)
    return g

g = unbroadcast(np.zeros((3, 1)), np.ones((2, 3, 4)))
print(g.shape)
print(g.ravel())
```

```
(3, 1)
[8. 8. 8.]
```

Answer

`F.shape = (100, 100)` because NumPy broadcasting copies `A` 100 times along axis 1 and `B` 100 times along axis 0. So `dl__dF.shape = (100, 100)`, but the gradients must match their inputs: `dl__dA.shape = (100, 1)` and `dl__dB.shape = (1, 100)`.

Why sum? Consider $f(y_1, y_2) = y_1 y_2$ where $y_1(x) = x$ and $y_2(x) = x$ are copies of the same variable. By the chain rule,

$$\partial_x f = \frac{\partial f}{\partial y_1}\partial_x y_1 + \frac{\partial f}{\partial y_2}\partial_x y_2 = \frac{\partial f}{\partial y_1} + \frac{\partial f}{\partial y_2}.$$

When a variable is copied and the copies are used in a function, the derivative is the **sum** of the derivatives with respect to the copies. Broadcasting creates copies along the mismatched axes, so `unbroadcast` sums the gradient along those axes:

```
dl__dA = dl__dF.sum(axis=1, keepdims=True)  # (100, 1)
dl__dB = dl__dF.sum(axis=0, keepdims=True)  # (1, 100)
```

The code prints

```
(3, 1)
[8. 8. 8.]
```

The extra leading axis is summed first, giving shape `(3, 4)` with value 2. Then axis 1 (size 1 in the target) is summed with `keepdims`, giving shape `(3, 1)` with value $2 \times 4 = 8$.